

Real FP4 Tensor-Core Code in Pure Rust on a Gaming GPU

— with NVIDIA’s Own Compiler

[Carter Richardson]
[48933813+botBehavior@users.noreply.github.com]

Abstract

We report a viability result: an entire Llama-class decoder, written in pure Rust and compiled to PTX by NVIDIA’s own experimental first-party Rust→PTX backend (`cuda-oxide`), runs FP4-quantized weights on a consumer NVIDIA RTX 5070 Ti and generates coherent English text. On a real TinyLlama-1.1B model quantized to MXFP4, the engine sustains roughly 181 tokens/s of decode throughput at steady state (median over a clean-GPU re-measurement; 188.6 tok/s in the original session, which the clean run reproduced) — within about 3.7×, or roughly 27%, of `llama.cpp`’s decode throughput (≈ 664 tok/s, Q4_K_M, hand-tuned CUDA with CUDA graphs) on the same GPU, on a first, unoptimized cut. Along the way we measured the actual programmable surface of consumer Blackwell (`sm_120`) for Rust kernels and found it richer than secondary sources imply: TMA, thread-block clusters with distributed shared memory, and native FP4 block-scaled tensor-core MMA all execute, while the `tcgen05` 5th-generation tensor-memory MMA pipeline correctly does not. We do **not** claim supremacy; `llama.cpp` is faster today, and we say where and why. The contribution is that the path exists end to end — a coherent LLM, compiled from Rust by NVIDIA’s compiler, on a gaming GPU — and that it lands close on the first attempt, with a clear optimization runway.

1 Introduction

The claim of this paper is narrow and we keep it narrow: **viability, not supremacy**.

We took NVIDIA’s experimental first-party Rust→PTX compiler, `cuda-oxide` (alpha), and used it to write every kernel of a Llama-class decoder in pure Rust — RMSNorm, RoPE, grouped-query flash decode, SwiGLU, and FP4-quantized matrix–vector projections — with no hand-written CUDA C++ in the inference path. We compiled those kernels to PTX through NVIDIA’s backend, loaded a real TinyLlama-1.1B model [?] quantized to 4-bit MXFP4, and generated text on a consumer RTX 5070 Ti.

The headline artifact is a single generation. Given the prompt “*The main advantage of running a neural network on a GPU instead of a CPU is*”, the engine continued:

...that it can handle larger datasets and faster processing speeds. This means that the model can learn more complex patterns and make more accurate predictions, which can lead to better results in real-world applications.

That is grammatical, on-topic, greedy-decoded text from a 4-bit-quantized LLM, produced entirely through NVIDIA’s Rust→PTX backend on a gaming GPU. It is 1.1B-parameter greedy output — coherent, not authoritative; we use it to demonstrate that the forward pass is numerically correct end to end, not as a source of fact.

Contributions.

1. A **measured capability map** of consumer Blackwell (`sm_120`, RTX 5070 Ti) for Rust GPU kernels (§4), correcting two “datacenter-only” assumptions from our pre-build research — one echoing a community Blackwell wiki, one our own conservative read — that measurement refutes (TMA and clusters/DSMEM both work), while confirming the expected datacenter exclusions (`tcgen05`/TMEM, WGMMMA, CTA-pair multicast).
2. A pinned, CPU-reference-validated **FP4 (NVFP4/MXFP4) m16n8k64 block-scaled tensor-core MMA** driven from pure Rust through `cuda-oxide` (§5) — the full element and scale-factor operand layout, which, to the best of the positioning in §3, is the first pure-Rust FP4 tensor-core MMA demonstrated on consumer Blackwell (we did not perform an exhaustive prior-art search; see the hedge in §5).
3. A complete **pure-Rust FP4 decode engine** (§6): per-kernel CPU-parity validation, an end-to-end forward pass on real weights, and coherent text from a real model.
4. An **honest head-to-head** against `llama.cpp` [?] on the same GPU and model (§6), where we lose on raw decode speed by about $3.7\times$, with a gap analysis that attributes the difference to specific, addressable engineering — not to the language or the compiler.

We are deliberate about framing throughout. Every speed claim carries its measurement and configuration; every novelty claim is hedged; and where a weak signal corroborates a strong conclusion, the two are stated as separate sentences with separate weight, never fused.

2 Background: `cuda-oxide` and consumer Blackwell

`cuda-oxide` [?] is NVIDIA’s experimental, alpha-stage compiler backend that lowers Rust (via a custom `rustc` codegen backend over LLVM) directly to PTX, NVIDIA’s virtual ISA, which `ptxas` then assembles to SASS for a specific GPU. The toolchain ships device intrinsics for the modern PTX feature set, a `cargo oxide` driver, and a set of example kernels. It is pre-1.0 and moves quickly; the results here are pinned to a specific commit (§A) and should be treated as a snapshot. The significance for this work is that the kernel source is ordinary `no_std` Rust — the same language as the host harness, with shared `#[repr(C)]` data layouts — rather than CUDA C++ with a Rust shim. The inference path contains no hand-written CUDA C++.

The RTX 5070 Ti is a GB203 part with compute capability 12.0 (`sm_120`). It is a Blackwell-generation consumer GPU, distinct from datacenter Blackwell (`sm_100`, e.g. B200). The two share the Blackwell numeric formats but differ in the tensor-core pipeline: datacenter Blackwell adds the 5th-generation tensor-memory (`tcgen05`/TMEM) MMA path and its associated speed-of-light GEMM, which consumer Blackwell does not have. A central question for any Rust-on-Blackwell effort is exactly *which* of the advertised Blackwell features are reachable on the consumer part; §4 answers it by measurement.

Blackwell tensor cores support 4-bit floating-point operands in the E2M1 format (1 sign, 2 exponent, 1 mantissa bit) with block-wise scale factors. **MXFP4** [?] uses 32-element blocks with a power-of-two (`ue8m0`) scale per block; **NVFP4** [?] uses 16-element blocks with a fractional (`ue4m3`) scale per block. Both are weight-only quantization targets here: the model’s projection weights are stored in FP4 with per-block scales, while activations remain higher precision. FP4 roughly quarters weight traffic relative to f16, which is the lever that matters for bandwidth-bound decode (§6).

feature	sm_120 result	evidence
atomics / sharedmem / warp-reduce / GEMM	executes	per-example PASS
TMA copy + pipeline	executes	4096/4096 match
clusters + DSMEM	executes	cross-block reduction PASSED
tcgen05 / tcgen05_matmul / gemm_sol	rejects	driver INVALID_PTX / requires sm_100
WGMMA	rejects	Hopper-only
TMA multicast	rejects	sm_100a

Table 1: Consumer Blackwell capability surface for Rust kernels, measured on an RTX 5070 Ti via `cuda-oxide` examples. TMA and clusters with distributed shared memory both execute as shipped examples here, and FP4 block-scaled tensor-core MMA is validated directly in §5 (no shipped example exercises it). Note that the CUDA-core GEMM examples emit FMA, not tensor-core MMA.

3 Related Work

Rust on GPUs. Several toolchains compile Rust toward GPU targets: `rust-gpu` [?] (SPIR-V/Vulkan), the Rust-CUDA project’s `rustc_codegen_nvvm` [?] (Rust→NVVM→PTX), and host-side binding and launch crates such as `cuda-arc` [?]; `ZLUDA` [?] re-targets CUDA binaries onto non-NVIDIA hardware. `cuda-oxide` differs in being NVIDIA’s own first-party experimental Rust→PTX backend, which is the toolchain whose consumer-Blackwell surface we measure.

FP4 / MXFP4 inference. Block-scaled 4-bit floating point — MXFP4 under the OCP microscaling specification [?] and NVFP4 in NVIDIA’s Blackwell tensor-core ISA [?] — is documented at the format and hardware level, but production FP4 kernels today are predominantly CUDA C++/PTX. We are not aware of a pure-Rust driver of the `m16n8k64` block-scaled FP4 MMA, which is the operand-layout contribution of §5.

Rust LLM inference engines. Rust inference stacks such as `candle` [?] run model forward passes from Rust but dispatch GPU compute through vendor kernels or CUDA C++ rather than compiling their kernels from Rust; `llama.cpp` [?] is our hand-tuned CUDA baseline (§6). The gap we fill is an end-to-end decoder whose every kernel is compiled from Rust by NVIDIA’s backend.

4 What executes on consumer sm_120 (and what correctly rejects)

Before building an engine we measured what actually runs. We compiled and executed a curated set of the backend’s own example kernels on the real 5070 Ti and classified each by its runtime log marker — a kernel counts as executing only if it emits an explicit correctness marker, with datacenter-target rejects and self-skips caught first. (The classifier was tightened after review so that it no longer matches PTX-generation boilerplate; it requires a genuine execution-success marker.) Table 1 summarizes the surface.

On the GEMM rows and tensor-core reachability. The shipped `gemm` and `tiled_gemm` examples computed correctly (2630 and 3530 GFLOPS, max relative error `1.5e-5`), but their PTX contains no `mma/wgmma`: these are CUDA-core FMA figures, not tensor-core peak, and we report them as such. The tensor cores themselves are reachable — a hand-written `mma.sync` assembles under `ptxas -arch=sm_120a` (cubin emitted), and §5 drives the FP4 block-scaled variant with real operands — but no shipped example exercises them, which is why we validate that path directly in isolation rather than inferring it from a stock benchmark.

On the tcgen05 rejection, two separate facts, kept separate. These corroborate each other but carry different weight, and we do not fuse them. The load-bearing fact is hardware: on a

real `sm_120` part the CUDA driver returns `INVALID_PTX` when loading a `tcgen05` module, because consumer Blackwell lacks the 5th-generation tensor-memory (`tcgen05`/TMEM) MMA path that datacenter `sm_100` adds [?]. That is *why* the feature does not run, and it is the claim. The corroborating, weaker, separate fact is toolchain: `cuda-oxide` *does* implement the `tcgen05` intrinsics and emits real `tcgen05.*` PTX (its codegen table even optimistically lists `sm_120/sm_120a`), and when *forced* to the `sm_120a` target the backend hits an LLVM `CannotYetSelect` on the instruction sequence — an instruction-selection gap at that target. A reviewer can open the backend source and see the codegen path present, so this is corroboration only. **We never claim the codegen path is absent**; the hardware reject is the conclusion, and the selection gap is a footnote to it.

The headline finding. The common shorthand “consumer Blackwell is just a cut-down datacenter Blackwell” is wrong in both directions. Consumer `sm_120` *does* run TMA, thread-block clusters with distributed shared memory, and native FP4 block-scaled tensor-core MMA — features frequently assumed datacenter-only — in one case contradicting a community Blackwell wiki’s “cluster size 1” claim, in the other overturning our own conservative pre-build assessment that TMA was probably unusable on the 5070 Ti. Our own pre-build research correctly nailed the consumer/-datacenter `tcgen05`/TMEM split but inherited two “datacenter-only” claims that measurement refutes: that TMA does not work usefully on the 5070 Ti (it does — `cp.async.bulk.tensor` and a TMA pipeline both run correctly), and that clusters are limited to size 1 (a $4 \times 1 \times 1$ cluster performing a cross-block DSMEM reduction passes). Consumer `sm_120` *does not* run the `tcgen05`/TMEM MMA pipeline, its speed-of-light GEMM, WGMMA, or CTA-pair TMA multicast — those are datacenter `sm_100a` or Hopper. So the programmable surface for Rust kernels on consumer Blackwell is full `mma.sync` tensor cores including FP4, TMA, clusters/DSMEM, `cp.async`, scoped atomics, `bf16x2`, and `dp4a` — everything except the `tcgen05`/TMEM datacenter MMA path. That surface is the substrate the engine in §6 is built on.

5 FP4 block-scaled MMA (m16n8k64) in Rust

The FP4 block-scaled MMA is the operand-layout-heavy part of the work, so we validated it in isolation before wiring it into the engine. The MXFP4 path uses, at the PTX level:

```
mma.sync.aligned.m16n8k64.row.col.kind::mxf4.block_scale.scale_vec::2X.f32
.e2m1.e2m1.f32.ue8m0
{d0,d1,d2,d3}, {a0,a1,a2,a3}, {b0,b1}, {c0,c1,c2,c3},
sa, {byte_id,thread_id}, sb, {byte_id,thread_id};
```

(NVFP4 uses `kind::mxf4nvf4`, `scale_vec::4X`, `.ue4m3`.) The operand fragments are $A = 4 \times .b32$, $B = 2 \times .b32$, $C/D = 4 \times .f32$, with 4-bit E2M1 values packed 8 per `.b32`. The scale operands `sa/sb` are `.b32` registers; the accompanying byte-id/thread-id are *immediate integer constants*, not registers — supplying them as registers was the source of an early `ptxas` parse error.

Target gate. The block-scaled MMA requires `.target sm_120a` (the architecture-*specific* variant). Plain `sm_120` rejects it with “Instruction ‘mma with block scale’ not supported on .target ‘sm_120’.” For `nvcc` this means `-gencode arch=compute_120a,code=sm_120a`; for `cuda-oxide`, build with `--arch sm_120a`.

Numerical validation. We validated correctness against a CPU reference at three levels, all with zero mismatches. An all-ones test ($A = B = 1.0$, $\text{scale} = 1.0$) yields every output element $= \sum_{k=0}^{63} 1 \cdot 1 = 64.0$; measured $128/128 = 64.0$. This proves the instruction executes, the E2M1 encoding and `ue8m0` scale decode correctly, and the $k = 64$ accumulation is right, without depending on the exact per-thread fragment mapping. The element fragment mapping was then pinned against non-uniform FP4 data with full CPU-reference parity (0/128 mismatch), recording the

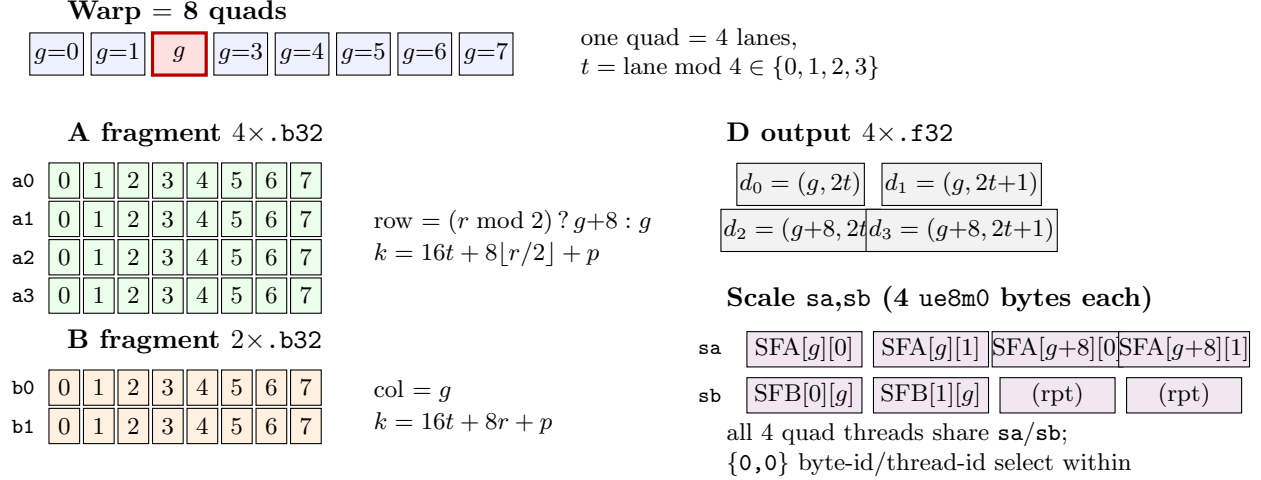


Figure 1: Single-quad fragment map for the m16n8k64 block-scaled FP4 MMA, drawn from the validated layout of Table 2. A warp’s 32 lanes split into 8 quads indexed $g = \lfloor \text{lane}/4 \rfloor$, each holding 4 lanes $t = \text{lane mod } 4$; one quad (highlighted) is expanded. Each .b32 register packs eight E2M1 values at nibble positions $p = 0 \dots 7$ (low nibble = lowest K), and r is the register index within a fragment. The A fragment ($a_0 \dots a_3$) and B fragment (b_0, b_1) give each value’s matrix row/column and K index; the D fragment ($d_0 \dots d_3$) gives the four output positions. All four threads of the quad carry the same scale registers **sa/sb** (four ue8m0 bytes each, ue8m0 byte = $127 + \log_2(\text{scale})$); the $\{0,0\}$ byte-id/thread-id immediates select within them.

operand	fragment	element position (nibble $p = 0 \dots 7$, register index r)
A	16×64 row-major, $a_0 \dots a_3$	row = $(r \bmod 2) ? g+8 : g$, $k = 16t + 8\lfloor r/2 \rfloor + p$
B	64×8 col-major, b_0, b_1	col = g , $k = 16t + 8r + p$
D	16×8 , $d_0 \dots d_3$	$(g, 2t), (g, 2t+1), (g+8, 2t), (g+8, 2t+1)$

Table 2: Pinned m16n8k64 E2M1 operand layout, recovered against a CPU reference (0/128 mismatch). Lane decomposes as $g = \lfloor \text{lane}/4 \rfloor$, $t = \text{lane mod } 4$; each .b32 packs eight E2M1 values, low nibble the lowest K .

exact m16n8k64 E2M1 thread/register layout. The scale-factor layout was pinned with non-uniform power-of-two scales (0/128 mismatch): `scale_vec::2X`, $K = 64 \rightarrow 2$ blocks/row, ue8m0 byte = $127 + \log_2(\text{scale})$. With this, a full FP4 GEMM with per-block scales is buildable.

The pinned layout. Table 2 records the recovered m16n8k64 thread/register map. Writing $g = \lfloor \text{lane}/4 \rfloor$ ($0 \dots 7$) and $t = \text{lane mod } 4$ ($0 \dots 3$), with the low nibble of each .b32 holding the lowest K index and eight E2M1 values packed per register, the element positions are as shown in Table 2 and Figure 1; the E2M1 magnitudes are $\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$ for code bits $0 \dots 2$ with bit 3 the sign ($1.0 = 0x2$). For the scale factors, `scale_vec::2X` with $K = 64$ gives two 32-element blocks per row, so each .b32 scale register holds four ue8m0 bytes: **sa** = [SFA[g][0], SFA[g][1], SFA[$g+8$][0], SFA[$g+8$][1]] and **sb** = [SFB[0][g], SFB[1][g], ...] in its low two bytes (replicated to the high two). All four threads of a quad carry the same **sa/sb**; the hardware selects within them via the $\{0,0\}$ byte-id/thread-id immediates, and the ue8m0 byte encodes the power-of-two scale as $127 + \log_2(\text{scale})$ ($1.0 = 0x7f$). Pinning this exact layout is what turns a single validated MMA into a buildable per-block-scaled GEMM, and it is the part a re-implementation would otherwise have to rediscover by trial against ptxas.

kernel	example	dims	result (max rel. err)
RMSNorm	<code>llama_rmsnorm</code>	dim 2048	7e-7, 0/2048
RoPE (rotate-half, HF)	<code>llama_rope</code>	32 heads $\times$ 64, $\theta=5e5$	1.4e-5, 0/2048
Flash-attn decode (GQA)	<code>llama_attn_decode</code>	32 q / 8 kv heads, seq 512	4.4e-5, 0/2048
FP4 decode GEMV	<code>fp4_gemv</code>	up to 16384 $\times$ 4096	$\sim 1.3e-3$, 0 bad

Table 3: Per-kernel CPU-parity validation. Every forward-pass kernel was gated against a CPU reference before assembly.

Driven from pure Rust. The MMA runs through `cuda-oxide`, not only through `nvcc`: the `fp4_mma` example produces $\checkmark$ all 128 = 64.0. Because the backend’s inline-asm macro allows only a single `out` operand, the MMA plus its four `st.global` stores live in one memory-clobbering inline-asm block writing through a per-lane pointer.

Novelty hedge. To the best of the positioning in §3, this is the first pure-Rust FP4 tensor-core MMA demonstrated on consumer Blackwell. We did *not* perform an exhaustive prior-art search, so we state this as a belief, not an established first.

6 End-to-end: TinyLlama-1.1B MXFP4 decode

Kernels, each CPU-parity validated. Every kernel in the forward pass was built in pure Rust and gated against a CPU reference before assembly (Table 3). Reusable patterns: block-reduce via warp shuffle plus shared-memory partials (RMSNorm, softmax); a precomputed rotary cos/sin cache passed in from the host (no device transcendentals on the hot path); device `libdevice` math where needed.

End-to-end assembly. The full forward pass runs with FP4 weights on every projection: RMSNorm $\rightarrow$ FP4 QKV $\rightarrow$ RoPE $\rightarrow$ GQA flash-decode $\rightarrow$ FP4 output projection $\rightarrow$ residual $\rightarrow$ RMSNorm $\rightarrow$ FP4 SwiGLU MLP $\rightarrow$ residual, repeated across layers, then a final norm and FP4 LM head (vocab 128,256), with a KV cache. On a Llama-1B configuration (ctx 512, 32 tokens timed, random weights) it sustains 4.51 ms/token = 221.6 tokens/s, producing finite logits and a valid argmax. This is a *pipeline-performance* number on random weights — the token IDs are meaningless; output quality is established separately below.

The bandwidth-bound-decode thesis. Single-token decode is dominated by streaming the model’s weights once per token, not by arithmetic. At a 896 GB/s memory ceiling on the 5070 Ti [?], the relevant metric for the FP4 GEMV (which carries the QKV, output, MLP, and LM-head projections) is the fraction of that ceiling it reaches. This is *why* the missing `tcgen05` datacenter MMA path barely hurts decode: decode does not need speed-of-light GEMM, it needs bandwidth. Figure 2 reports measured FP4 GEMV weight-streaming bandwidth. Three measured lessons, stated as measured rather than assumed: a `const [f32;8]` decode LUT lowers to local memory under this backend, so decoding E2M1 arithmetically is faster; the backend did *not* vectorize a `repr(C) 4 $\times$ u32` struct load to `ld.global.v4` (it stayed scalar `b32`), and fewer-larger iterations *hurt* throughput by reducing memory-level parallelism, so the 16-iteration `u32` stream beat the 4-iteration 128-bit version; and staging the activation vector in shared memory once per block removes the $N \times$ re-read. The climb from 30% to 41% with size indicates the small-matrix figure is a launch/occupancy ramp, not kernel inefficiency. Notably, three of the capability surprises of §4 — TMA, `cp.async`, and clusters with distributed shared memory — are exactly the weight-streaming levers this kernel does *not* yet use: it streams with plain scalar `ld.global` and stages

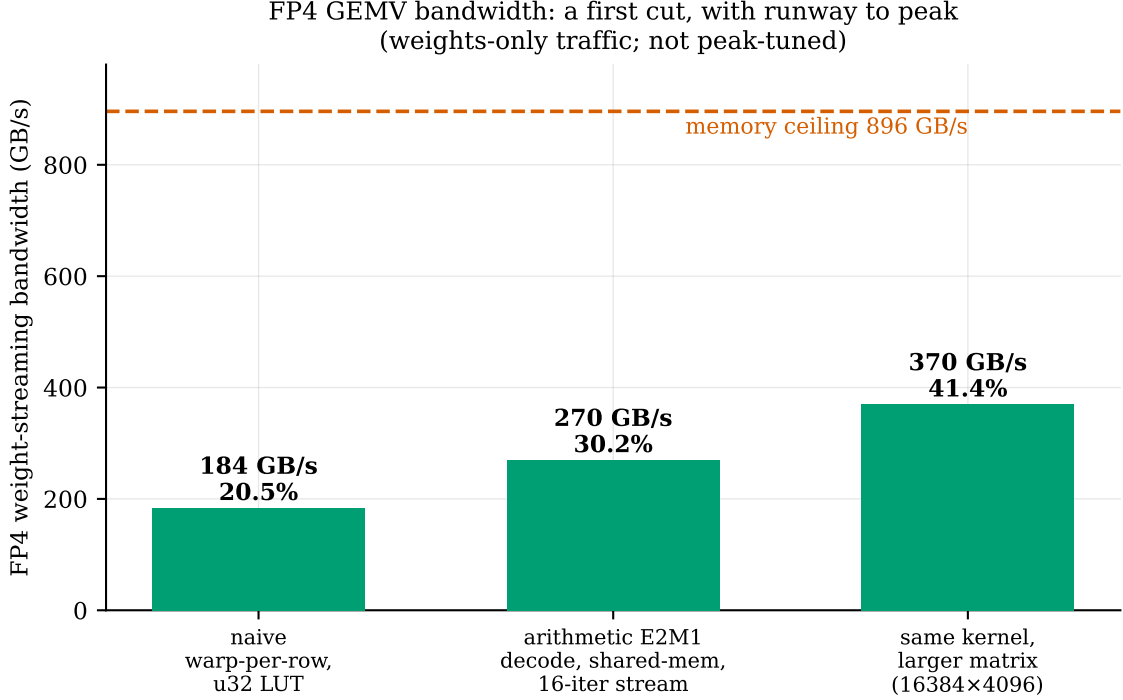


Figure 2: FP4 GEMV weight-streaming bandwidth (weights-only, the FP4-shrunk traffic), a first cut, not peak-tuned: from a naive u32-LUT kernel (~ 184 GB/s, 20.5% of the 896 GB/s ceiling), to arithmetic E2M1 decode with a shared-memory activation stream (270 GB/s, 30.2%), to 41.4% on a larger matrix. The climb with size indicates a launch/occupancy ramp, not kernel inefficiency.

activations in ordinary shared memory. Async-copy (`cp.async/TMA`) pipelining, hand-vectorized shared-memory loads (the backend would not emit `ld.global.v4` on its own), and multi-row-per-warp scheduling are the named, measured-as-available levers toward peak, left to dedicated perf work.

A real model: coherent text from TinyLlama-1.1B. The engine loads the real TinyLlama-1.1B safetensors [?] (bf16 $\rightarrow$ f32 on parse), quantizes every projection to MXFP4 (E2M1 with a per-32 absmx f32 scale) on the host, and performs embedding lookup, prefill, and greedy generation with device argmax (no per-token logit copy to the host). Tokenization (encode/decode) is a standalone Hugging Face `tokenizers cargo` binary kept out of the `cuda-oxide` build, so the compiled inference path stays purely the kernels. The prompt quoted in §1 produced the continuation shown there, and other prompts in the same domain give similarly coherent output. Model load and host quantization take ~ 22 s; generation runs at 188.6 tokens/s in the original session, reproduced on a clean GPU at steady state (below). This establishes what the random-weight run could not: safetensors load is correct, MXFP4 weight-only quantization preserves output quality, the full forward pass is correct on real weights, and end-to-end generation works.

Head-to-head with llama.cpp. We compared against `llama.cpp` [?] on the same GPU and the same model, both at 4-bit. `llama.cpp` was built from source against the full CUDA 12.9 toolkit (the 13.1 toolkit on this machine lacks cuBLAS; 12.9 is the full one and its `ptxas` supports `sm_120a`), with `CMAKE_CUDA_ARCHITECTURES=120`, and measured with `llama-bench` on TinyLlama-1.1B Q4_K_M, all layers on GPU. To avoid single-run artifacts, we re-measured both engines on an *uncontended* GPU in the same session, at steady state — timing 200 generated tokens, enough

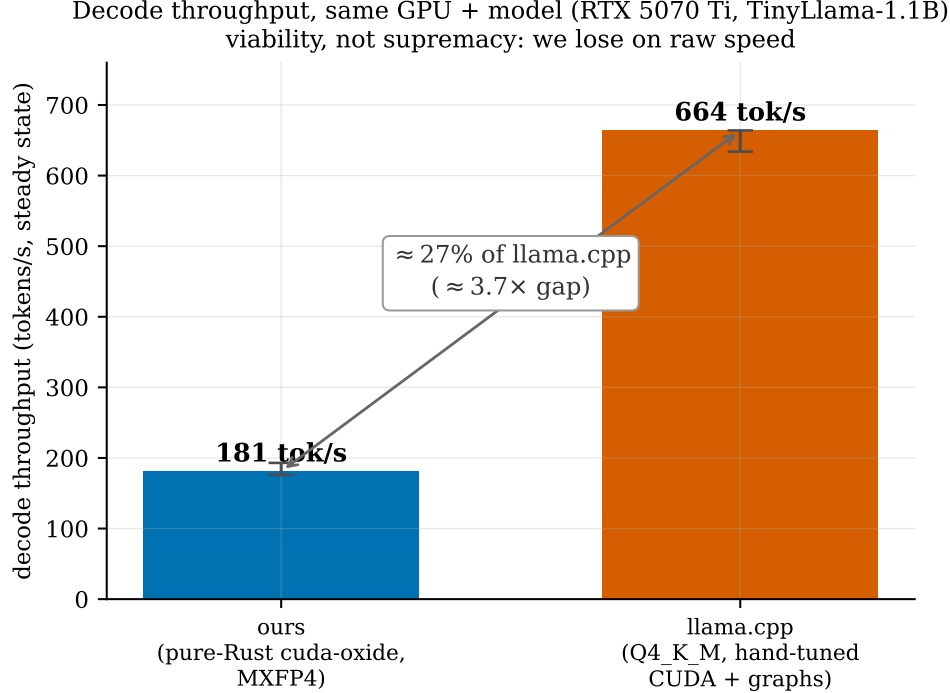


Figure 3: Decode head-to-head, steady-state on an uncontended GPU (TinyLlama-1.1B, 4-bit, RTX 5070 Ti): ours ≈ 181 tok/s (median; 188.6 original session) versus llama.cpp ≈ 664 (634–664; 633.9 original)—about 27% of its decode throughput, a $\approx 3.7\times$ gap. The honest result on a first, unoptimized cut; we make no supremacy claim.

to clear the GPU’s post-idle clock-ramp. Ours held a tight 176–193 tok/s (median 181 over five runs) and llama.cpp 634–664 tok/s (≈ 664), reproducing the original-session figures of 188.6 and 633.9. We report the reproducible clean-GPU medians: **ours ≈ 181 vs llama.cpp ≈ 664 — roughly $3.7\times$, about 27% of its decode throughput** (Figure 3). A short timing window, by contrast, reads our engine low and noisy because greedy generation finishes before the GPU clock fully boosts after the host-bound model load; llama-bench avoids this with dedicated warmup runs. The figures throughout are steady-state.

Fairness caveats, stated explicitly. The comparison is close but not apples-to-apples. Quantization differs: ours is MXFP4 (E2M1, per-32 power-of-two scale), theirs is Q4_K_M, a different 4-bit scheme, and we have not normalized for any quality or layout difference. The measurement harness differs: our figure is short-context decode, theirs is llama-bench’s tg128; we report decode throughput on both, but the exact harness differs. And we have no prefill path: llama.cpp prefill (pp512) reaches ~ 30 – 33 k tok/s using tensor cores, while we measure decode only — that is our chosen lane, and it means we do not compete on prefill at all. We make **no supremacy claim**. We lose on raw speed today; the point of the comparison is the magnitude of the gap on a first attempt, which the next section accounts for.

7 Limitations

The $\approx 3.7\times$ gap is explained by specific, addressable engineering, not by the language or the compiler. The engine issues roughly 200 unfused kernel launches per token, where llama.cpp

uses CUDA graphs to drive launch overhead to near zero — kernel fusion and/or CUDA graphs are the single largest expected win. Our FP4 GEMV runs at 30–41% of the 896 GB/s ceiling, where a tuned engine runs near peak; vectorized shared-memory loads, async-copy pipelining, and multi-row-per-warp scheduling are the levers. We use f32 activations, where `llama.cpp` uses f16, halving activation traffic and enabling faster math. Adding a compute-bound prefill path would broaden the comparison, though prefill is the regime where consumer Blackwell loses the datacenter speed-of-light GEMM, so we expect it to be the weaker lane and have not chased it. We have not isolated each lever’s contribution to the $\approx 3.7\times$: we order them by expected impact from the measured behavior — the roughly 200 unfused launches per token, the GEMV’s 30–41% fraction of the bandwidth ceiling, and the f32 activation width — not from a per-lever ablation we did not run. The three are also independent, so the wins are expected to compound rather than overlap.

The honest standing edges: the engine is decode-only and bandwidth-bound by design; the datacenter speed-of-light GEMM path is simply unavailable on consumer Blackwell; we measured a single GPU and SKU; and `cuda-oxide` is alpha and pre-1.0, so these results are a snapshot of a moving toolchain. The novelty claim of §5 is hedged: as far as we found, the FP4 MMA is a first, but we did not search prior art exhaustively. We exclude one earlier low reading (a 156 tok/s figure) from the results entirely; it was a measurement artifact from timing only 48 generated tokens during the GPU’s post-idle clock-ramp, fixed by timing 200 tokens at steady state, and we note it only as a methodology lesson. None of these edges is a dead end — the runway above is concrete and ordered by expected impact.

8 Conclusion

We did not set out to beat `llama.cpp`, and we did not. We set out to find whether NVIDIA’s own experimental Rust→PTX backend can compile a real, FP4-quantized LLM decoder, written entirely in pure Rust, into kernels that run on a consumer GPU and generate coherent text — and it can. Along the way we measured the actual Rust-programmable surface of consumer Blackwell and found it richer than the secondary sources said: TMA, clusters with distributed shared memory, and native FP4 block-scaled tensor-core MMA all execute, while the `tcgen05` datacenter pipeline correctly does not. The first cut lands within about $3.7\times$ of the hand-tuned `llama.cpp` baseline on the same hardware and model, and the gap is accounted for by ordinary, ordered optimization work — kernel fusion, GEMV tuning, f16 activations — not by anything fundamental to Rust or the compiler. The substrate is real, and the path is end to end. That is the result: viability, measured, with the edges named.

A Reproducibility

Hardware. RTX 5070 Ti (GB203, `sm_120`, compute capability 12.0), driver 596.49, under WSL2 Ubuntu 24.04. **Toolchain.** `cuda-oxide` v0.2.1 (built from source, commit 7dbd192), Rust `nightly-2026-04-03`, bundled LLVM 22.1.2 and clang-21. The FP4 block-scaled MMA requires building with `--arch sm_120a`; plain `sm_120` rejects the instruction. **CUDA toolkits — note the split.** The `cuda-oxide` PTX path used the machine’s CUDA 13.1 `ptxas` (which supports `sm_120a`); the `llama.cpp` baseline was built against CUDA 12.9, because the 13.1 install lacks cuBLAS while 12.9 is the full toolkit (its `ptxas` also supports `sm_120a`). These are two different uses of two different toolkits; we do not conflate them. **Measurement conditions.** Decode throughput is measured at steady state on an uncontended GPU: the box has two cards and host-side processes may share one, so we pinned both engines to the idle card and timed 200 tokens to clear the post-idle clock-ramp; each generation

follows a ~ 22 s host-bound model load, so short timing windows read our engine low. No clock throttling was observed. We only compare engines measured back-to-back in the same session.

References